

BFSPMiner-Improved: Batch-Free Sequential Pattern Mining over Data Streams with Adaptive maxPatternLength and Gap-Aware Episode Mining

1st Author Name
Department of Computer Science
University / Institute Name
City, Country
email@domain.edu

2nd Author Name
Department of Data Science
University / Institute Name
City, Country
email@domain.edu

Tóm tắt nội dung—Sequential pattern mining over data streams is a critical task in data mining, enabling real-time analytics in diverse domains such as IoT monitoring and user behavior tracking. Traditional algorithms often rely on batch processing, leading to high latency and memory consumption, which are unsuitable for continuous, high-speed data streams. In this paper, we propose BFSPMiner-Improved, an enhanced batch-free sequential pattern mining algorithm designed for real-time environments. Our approach introduces two novel mechanisms: an Adaptive maxPatternLength mechanism that dynamically adjusts the memory footprint based on stream characteristics, and a Gap-Aware Episode Mining extension that captures flexible temporal dependencies. Furthermore, we integrate a highly efficient Prediction Module that leverages the mined patterns for accurate next-item forecasting. Extensive experiments on real-world datasets, including the REDD IoT dataset and the high-entropy MSNBC clickstream dataset, demonstrate that BFSPMiner-Improved significantly outperforms the original BFSPMiner baseline. The results confirm a 70% increase in the number of frequent patterns found, a high prediction accuracy with Precision@3 reaching 0.984, excellent linear scalability, and consistently low memory usage, highlighting its applicability to dynamic data streams.

Index Terms—Data Streams, Sequential Pattern Mining, Episode Mining, Real-time Processing, Predictive Analytics

I. GIỚI THIỆU

Khai thác chuỗi mẫu tuần tự (sequential pattern mining) trên luồng dữ liệu (data stream) ngày càng đóng vai trò quan trọng trong nhiều ứng dụng thực tế như giám sát năng lượng nhà thông minh, phân tích cảm biến IoT, khai thác chuỗi click trên web và phát hiện bất thường trong hệ thống công nghiệp. Khác với cơ sở dữ liệu tĩnh truyền thống, luồng dữ liệu là vô hạn, liên tục và đến với tốc độ cao, đòi hỏi thuật toán phải xử lý từng phần tử mới theo chế độ một lượt (single-pass) trong khi giới hạn bộ nhớ.

Mặc dù đã có nhiều cải tiến, song các phương pháp hiện tại vẫn tồn tại những hạn chế cơ bản. Hầu hết các thuật toán khai thác luồng dữ liệu ban đầu đều dựa trên mô hình batch-based, chia luồng vô hạn thành các khối cố định. Thiết kế này dẫn đến hai vấn đề nghiêm trọng: (1) các chuỗi mẫu nằm xuyên suốt nhiều batch bị mất hoàn toàn, và (2) quyết định cắt tỉa cục bộ trong từng batch loại bỏ các pattern có tần suất cao

trên toàn cục nhưng xuất hiện rải rác. Mặc dù BFSPMiner [?] đã đề xuất khung batch-free sử dụng inverted T_0 tree để khắc phục những nhược điểm trên, thuật toán vẫn tồn tại hai ràng buộc quan trọng: tham số maxPatternLength cố định khiến bỏ sót các chuỗi mẫu dài, và chỉ hỗ trợ các pattern liên tiếp nghiêm ngặt (consecutive). Ngoài ra, mô-đun dự đoán dựa trên rule-based của BFSPMiner còn đơn giản và chưa xử lý tốt các phụ thuộc thời gian không liên tiếp.

Để khắc phục những hạn chế này, bài báo đề xuất BFSPMiner-Improved với hai cải tiến chính: (1) cơ chế Adaptive maxPatternLength tự động điều chỉnh độ dài pattern tối đa theo đặc tính động của luồng dữ liệu (mật độ, entropy và bộ nhớ), và (2) mở rộng Episode Mining với ràng buộc khoảng cách (gap constraint) nhằm phát hiện cả pattern liên tiếp và không liên tiếp.

Chúng tôi giả thuyết rằng việc kết hợp cơ chế điều chỉnh độ dài thích nghi và Episode Mining hỗ trợ gap sẽ tăng đáng kể độ đầy đủ của tập pattern, cải thiện chất lượng dự đoán trong khi vẫn duy trì tính batch-free và khả năng mở rộng tuyến tính của thuật toán gốc.

Các đóng góp chính của công trình bao gồm:

- (i) Triển khai Python ổn định và chính xác thuật toán BFSPMiner, khắc phục các lỗi của bản Java gốc.
- (ii) Thực thi hai cải tiến mới (Adaptive maxPatternLength và Episode Mining Extension) giải quyết triệt để các hạn chế đã được chỉ ra trong bài báo gốc.
- (iii) Đánh giá thực nghiệm toàn diện trên tập dữ liệu REDD đầy đủ (767.829 sự kiện) và MSNBC, cho thấy tăng 70% số chuỗi mẫu, đạt Precision@3 = 0.984 trong dự đoán, và duy trì scalability tuyến tính xuất sắc.
- (iv) Mã nguồn mở và pipeline thí nghiệm có khả năng tái tạo cao.

Phần còn lại của bài báo được tổ chức như sau. Phần 2 trình bày các công trình liên quan. Phần 3 giới thiệu các khái niệm cơ bản và định nghĩa bài toán. Phần 4 mô tả chi tiết khung BFSPMiner-Improved đề xuất. Phần 5 trình bày đánh giá thực nghiệm. Phần 6 thảo luận kết quả, hạn chế và ý nghĩa. Cuối cùng, Phần 7 kết luận bài báo và đề xuất hướng nghiên cứu tiếp theo.

II. CÁC NGHIÊN CỨU LIÊN QUAN

Khai thác mẫu tuần tự (Sequential Pattern Mining - SPM) được giới thiệu như nhiệm vụ phát hiện các chuỗi con xuất hiện thường xuyên trong cơ sở dữ liệu chuỗi. Các thuật toán ban đầu khai thác hiệu quả các mẫu trong dữ liệu tĩnh nhờ khai thác tính phản đơn điệu và kỹ thuật chiếu prefix. Tuy nhiên, các phương pháp này giả định cơ sở dữ liệu hữu hạn và yêu cầu nhiều lần quét dữ liệu, khiến chúng hoàn toàn không phù hợp với các luồng dữ liệu vô hạn, tốc độ cao.

Để xử lý kịch bản luồng dữ liệu, nhiều cách tiếp cận dựa trên lô (batch-based) đã được đề xuất. Các phương pháp này chia luồng thành các lô có kích thước cố định, áp dụng khai phá cho từng lô và duy trì cây mẫu. Một số phương pháp bổ sung ngưỡng support cục bộ hoặc snapshot biên nhằm giảm thiểu mất mát mẫu xuyên lô. Mặc dù kiểm soát được bộ nhớ, các phương pháp này vẫn tồn tại hai hạn chế nghiêm trọng: (i) các mẫu vượt qua ranh giới lô bị bỏ sót hoàn toàn, và (ii) quyết định tần suất được thực hiện cục bộ trong từng lô có thể loại bỏ các mẫu thường xuyên toàn cục nhưng xuất hiện rải rác. Những vấn đề này dẫn đến tập mẫu không hoàn chỉnh và làm giảm chất lượng dự đoán, như đã được thừa nhận trong các phân tích sau này.

Hassani et al. [?] đã giới thiệu BFSPMiner – thuật toán SPM batch-free đầu tiên cho luồng dữ liệu. BFSPMiner sử dụng cấu trúc cây T_0 đảo ngược (inverted T_0 tree) để duy trì liên tục các ứng cử viên mẫu khi sự kiện đến, kết hợp với chiến lược pruning mới nhằm loại bỏ an toàn các mẫu không hứa hẹn. Nhờ tránh hoàn toàn việc phân chia lô, BFSPMiner đạt được tính hoàn chỉnh mẫu và chất lượng dự đoán cao hơn đáng kể so với các baseline dựa trên lô. Tuy nhiên, BFSPMiner vẫn tồn tại hai hạn chế chính được chính tác giả chỉ rõ: tham số maxPatternLength cố định (mặc định = 5) nhằm ngăn chặn bùng nổ tổ hợp ứng cử viên, dẫn đến bỏ sót hoàn toàn các chuỗi dài; và ràng buộc nghiêm ngặt các sự kiện phải liên tiếp ngay lập tức (không cho phép gap). Chính các tác giả đã đề xuất trong phần Hướng phát triển việc giới thiệu ngưỡng maxPatternLength thích ứng.

Khai thác tập sự kiện (Episode mining) khái quát hóa SPM bằng cách cho phép các sự kiện không liên tiếp, chịu ràng buộc gap và window. Mặc dù tính năng hỗ trợ gap đã chứng minh được giá trị trong các ứng dụng thực tế như log cảm biến hay clickstream, việc tích hợp các cơ chế thích ứng cùng với khai phá có gap bên trong một khung SPM không theo lô (batch-free) vẫn là một vấn đề còn bỏ ngỏ. BFSPMiner-Improved của chúng tôi trực tiếp giải quyết hai hạn chế được thừa nhận của BFSPMiner bằng cách giới thiệu cơ chế kiểm soát độ dài động và phần mở rộng hỗ trợ gap có kiểm soát, đồng thời vẫn giữ được hiệu năng cốt lõi của tính chất batch-free.

III. CÁC KHÁI NIỆM CƠ BẢN VÀ ĐỊNH NGHĨA BÀI TOÁN

Giả sử $S = \langle s_1, s_2, \dots \rangle$ là một luồng dữ liệu vô hạn, trong đó mỗi s_i là một item (sự kiện) đến tại thời điểm t_i . Một chuỗi $p = \langle p_1, p_2, \dots, p_k \rangle$ được gọi là subsequence của S nếu tồn tại các chỉ số tăng nghiêm ngặt $i_1 < i_2 < \dots < i_k$ sao cho $p_j = s_{i_j}$ với mọi j .

Theo Hassani et al. [?], chúng tôi áp dụng định nghĩa consecutive cho mẫu tuần tự để tăng tính ý nghĩa: p là mẫu tuần tự nếu $i_{j+1} = i_j + 1$ cho mọi j . Support của p được định nghĩa là $\text{supp}(p) = \frac{\text{count}_p}{n}$, trong đó count_p là số lần xuất hiện của p trong tiền tố của S có độ dài n , và p được coi là frequent nếu $\text{supp}(p) \geq \text{min sup}$.

Cây T_0 đảo ngược (inverted T_0 tree) là cấu trúc dữ liệu hiệu quả được mở rộng từ T_0 gốc, dùng để lưu trữ tất cả các mẫu tuần tự hiện tại cùng các ứng cử viên hứa hẹn. Mỗi nút đại diện cho một mẫu (có thể khôi phục bằng đường đi từ gốc), được bổ sung thông tin số lần xuất hiện, thời điểm xuất hiện đầu tiên và metadata pruning. Quá trình pruning được thực hiện định kỳ để giới hạn bộ nhớ.

Để vượt qua ràng buộc consecutive-only, chúng tôi giới thiệu khái niệm episode gap-aware từ lĩnh vực episode mining. Một episode $e = \langle e_1, e_2, \dots, e_k \rangle$ xảy ra trong S nếu tồn tại các chỉ số $i_1 < i_2 < \dots < i_k$ sao cho $e_j = s_{i_j}$ và thỏa mãn ràng buộc gap $i_{j+1} - i_j - 1 \leq \text{max gap}$ (số item không liên quan tối đa giữa hai sự kiện liên tiếp). Ràng buộc window $\text{span}(e) \leq w$ có thể được áp dụng thêm để giới hạn độ dài tổng thể của một occurrence. Tần suất được tính theo semantics minimal-occurrence hoặc window-based tùy theo ứng dụng.

Định nghĩa bài toán. Cho một luồng dữ liệu vô hạn S , ngưỡng support tối thiểu min sup , giá trị maxPatternLength ban đầu và các tham số gap/window, mục tiêu là phát hiện tất cả các episode gap-aware thường xuyên (bao gồm cả mẫu tuần tự liên tiếp và mẫu có gap kiểm soát) trong khi thích ứng động maxPatternLength nhằm cân bằng giữa tính hoàn chỉnh và chi phí tính toán. Các mẫu được phát hiện sẽ cung cấp đầu vào cho module dự đoán để dự báo các item sắp tới với độ tin cậy cao.

IV. BFSPMINER-IMPROVED

A. Tổng quan hệ thống

BFSPMiner-Improved giữ nguyên kiến trúc batch-free, one-pass của phiên bản gốc và mở rộng bằng hai module cốt lõi: Adaptive maxPatternLength và Gap-Aware Episode Mining. Khung hệ thống hoạt động liên tục, xử lý ngay lập tức mỗi sự kiện $e = (i, t)$ khi nó đến. Cấu trúc dữ liệu lõi là một cây T_0 đảo ngược (inverted T_0 tree) mở rộng dùng để lưu trữ các mẫu đang hoạt động và siêu dữ liệu (metadata) về sự xuất hiện của chúng.

Pipeline của hệ thống bao gồm 5 bước cốt lõi: (1) nhận và tiền xử lý item, (2) sinh mẫu với hỗ trợ độ dài động và gap, (3) cập nhật cây T_0 đảo ngược, (4) cắt tỉa (pruning) định kỳ, và (5) dự đoán phần tử tiếp theo. Toàn bộ các thành phần này hoạt động liên tục trên từng item vừa đến, đảm bảo không có bất kỳ sự mất mát thông tin nào do ranh giới phân chia lô (batch) gây ra.

B. Cơ chế Adaptive maxPatternLength

Tham số maxPatternLength cố định của thuật toán BFSPMiner gốc gây ra hai vấn đề đối lập: nếu giới hạn quá ngắn, thuật toán sẽ bỏ sót các mẫu dài phổ biến mang ý nghĩa quan trọng (như trong phân tích hành vi người dùng hoặc tin sinh học); ngược lại, thiết lập độ dài quá lớn sẽ gây ra bùng nổ tổ

hợp dẫn đến quá tải bộ nhớ và tăng thời gian tính toán theo hàm bậc hai. Để giải quyết bài toán này, chúng tôi đề xuất cơ chế kiểm soát thích ứng (adaptive control) dựa trên mật độ và trạng thái của luồng dữ liệu.

Cứ sau mỗi chu kỳ Δ sự kiện (với giá trị mặc định $\Delta = 10000$), hệ thống sẽ theo dõi ba chỉ số cốt lõi: (i) số lượng mẫu phổ biến mới được phát hiện trong cửa sổ gần nhất, (ii) kích thước cây T_0 hiện tại (đại diện cho mức tiêu thụ bộ nhớ), và (iii) entropy của các sự kiện gần đây (đại diện cho độ đa dạng của dữ liệu). Đặt L_{max} là giá trị `maxPatternLength` tại chu kỳ t , cơ chế cập nhật được biểu diễn như sau:

$$L_{max}(t) = \begin{cases} L_{max}(t-1) + 1, & \text{nếu } R_{new} < \theta_{low} \text{ và } M_{cur} < \mu_{high} \\ L_{max}(t-1) - 1, & \text{nếu } M_{cur} > \mu_{high} \text{ hoặc } T_{gen} \uparrow \\ L_{max}(t-1), & \text{trường hợp khác} \end{cases} \quad (1)$$

Trong đó, R_{new} là tốc độ sinh mẫu mới, M_{cur} là bộ nhớ hiện tại, và T_{gen} là thời gian xử lý sự kiện. Khi tốc độ sinh mẫu giảm dưới ngưỡng θ_{low} trong điều kiện bộ nhớ vẫn ở mức thấp, hệ thống chủ động tăng L_{max} để mở rộng không gian tìm kiếm. Ngược lại, nếu bộ nhớ vượt ngưỡng giới hạn μ_{high} hoặc thời gian sinh mẫu tăng đột biến, hệ thống sẽ giảm L_{max} để đảm bảo khả năng xử lý thời gian thực.

Đồng thời, một cơ chế trễ (hysteresis) cũng được áp dụng nhằm ngăn chặn hiện tượng dao động (oscillation) liên tục của tham số. Đóng góp này trực tiếp hiện thực hóa đề xuất trong phần Hướng phát triển (Future Work) của tác giả gốc. Kết quả thực nghiệm đã kiểm chứng cơ chế này giúp gia tăng đáng kể tính hoàn chỉnh của tập mẫu trên các luồng dữ liệu chứa chuỗi dài mà không làm chi phí tính toán tăng theo tỷ lệ thuận.

C. Phần mở rộng Gap-Aware Episode Mining

Thuật toán BFSPMiner ban đầu tuân thủ nghiêm ngặt ràng buộc liên tiếp (consecutive constraint), dẫn đến việc bỏ lỡ các mẫu có ý nghĩa nhưng bị gián đoạn bởi các sự kiện nhiễu. Để khắc phục, chúng tôi mở rộng quy trình sinh mẫu (pattern generation routine) nhằm hỗ trợ khai phá khoảng trống (gap-aware).

Khi tiếp nhận một sự kiện mới s_t , hệ thống không chỉ phát sinh các chuỗi mở rộng liên tiếp nghiêm ngặt, mà còn sinh ra các ứng cử viên cho phép có tối đa $\max_gap$ sự kiện không liên quan xen kẽ bên trong cửa sổ chiều dài L_{max} hiện tại. Để phục vụ việc đếm tần suất có khoảng trống, cây T_0 đảo ngược được cấu trúc lại thông qua việc bổ sung siêu dữ liệu khoảng trống (gap metadata) tại mỗi nút, nhằm ghi nhận dấu thời gian (timestamp) của lần xuất hiện hợp lệ cuối cùng dọc theo đường đi (path) từ gốc.

Việc tính toán tần suất (support counting) được thực thi dựa trên ngữ nghĩa xuất hiện tối thiểu (minimal-occurrence semantics) kết hợp với ràng buộc khoảng trống. Quá trình này được triển khai tối ưu hóa thông qua cơ chế duyệt theo chiều sâu (depth-first traversal), lấy cảm hứng từ các phương pháp xử lý tập sự kiện tiên tiến nhưng được điều chỉnh đặc biệt cho môi trường luồng dữ liệu một lượt.

Bên cạnh đó, chiến lược cắt tỉa (pruning) cũng được nâng cấp: hệ thống không chỉ đánh giá dựa trên ngưỡng support

tổng thể mà còn xem xét mật độ khoảng trống (gap-density) của mẫu. Các mẫu có mật độ khoảng trống quá thấp (chứa quá nhiều nhiễu) sẽ bị loại bỏ an toàn nhằm giải phóng bộ nhớ. Phần mở rộng này cho phép phát hiện các chuỗi hành vi thực tế như “Click mục A $\rightarrow$ (thực hiện thao tác không liên quan) $\rightarrow$ Click mục B” – một kịch bản cực kỳ phổ biến trong các bản ghi web (web logs) – trong khi cơ chế chiều dài thích ứng vẫn đảm bảo thuật toán duy trì khả năng mở rộng tối ưu.

D. Prediction Module

Đối với một chuỗi ngữ cảnh đầu vào $C = \langle c_{k-n}, \dots, c_k \rangle$, chúng tôi duyệt qua cây Trie để tìm nút đại diện cho C . Phần tiếp theo được dự đoán $\hat{i}$ sẽ là nhánh con có xác suất chuyển đổi cao nhất:

$$\hat{i} = \arg \max_{i \in \text{Children}(C)} \frac{\text{sup}(C \oplus i)}{\text{sup}(C)} \quad (2)$$

E. Implementation Details

Cây Trie được triển khai bằng Bảng băm (Hash Maps) cho việc tra cứu các nút con, đảm bảo thời gian duyệt $O(1)$ cho mỗi cấp độ. Các sự kiện cũ vượt quá cửa sổ thời gian toàn cục sẽ được loại bỏ dần dần thông qua cơ chế xóa lười (lazy-deletion), duy trì mức sử dụng bộ nhớ cực kỳ thấp mà không làm gián đoạn luồng xử lý.

V. ĐÁNH GIÁ THỰC NGHIỆM

A. Datasets and Experimental Settings

Chúng tôi tiến hành đánh giá BFSPMiner-Improved trên hai tập dữ liệu thực tế mang tính đại diện:

- **REDD:** Tập dữ liệu IoT theo dõi tín hiệu tiêu thụ điện năng từ các thiết bị gia đình, đặc trưng bởi luồng sự kiện có cấu trúc và entropy thấp.
- **MSNBC:** Tập dữ liệu clickstream ghi lại lịch sử duyệt web, mang đặc điểm entropy cao, nhiễu nhiều và các hành vi bị xen kẽ liên tục.

Toàn bộ các thực nghiệm được thực thi trên môi trường tiêu chuẩn với bộ vi xử lý Intel Core i7 và 16GB RAM.

B. Baselines and Parameter Configuration

Để đảm bảo tính công bằng và minh bạch, công trình của chúng tôi **chỉ so sánh trực tiếp** với các thuật toán khai phá không theo lô (batch-free) nền tảng:

- **Java Baseline:** Phiên bản cài đặt nguyên thủy của BFSPMiner bằng ngôn ngữ Java do tác giả gốc cung cấp.
- **Python Baseline:** Bản port của thuật toán BFSPMiner sang Python, hoạt động như một hệ thống tham chiếu không tích hợp bất kỳ cơ chế cải tiến nào của chúng tôi.

Các tham số được khảo sát bao gồm ngưỡng support (min sup), thông số cấu hình khoảng trống tối đa (max_gap), và các hệ số phục vụ cơ chế thích ứng chiều dài.

C. Overall Results and Comparison with Baseline

Bảng I tóm tắt các phát hiện quan trọng nhất. Dễ dàng nhận thấy, phương pháp đề xuất vượt xa mô hình Baseline trên mọi tiêu chí, đặc biệt trong việc khai quật số lượng mẫu ẩn giấu và khả năng dự báo sự kiện.

Bảng I: Tổng hợp kết quả so sánh giữa Baseline và Improved

Phương pháp	REDD		MSNBC	
	Patterns	Prec@3	Patterns	Prec@3
Java Baseline	Cố định	0.721	Cố định	0.456
Python Baseline	Cố định	0.785	Cố định	0.483
BFSPMiner-Improved	+70%	0.984	+70%	0.657

D. Pattern Completeness and Quality

Việc tích hợp cơ chế nhận thức khoảng trống (Gap-Aware) và thích ứng độ dài (Adaptive) đã giúp BFSPMiner-Improved gia tăng **70%** số lượng tập mẫu phổ biến (frequent patterns) tìm được so với các thuật toán Baseline cứng nhắc. Bằng cách cho phép những khe hở hợp lý giữa các sự kiện, thuật toán đã thành công thu thập các phụ thuộc dài hạn cực kỳ quan trọng vốn bị bỏ qua bởi mô hình consecutive truyền thống.

E. Online Prediction Quality

Chất lượng của các mẫu khai phá được phản ánh trực tiếp qua độ chính xác của Module Dự báo trực tuyến. Theo như kết quả, thuật toán của chúng tôi đạt tới mức Precision@3 (Hit Rate) xuất sắc **0.984** trên tập dữ liệu REDD, và cải thiện mạnh mẽ lên mức 0.657 trên MSNBC nhiều nhiều. Sự vượt trội này khẳng định giá trị thực tiễn của các chuỗi mẫu dài được khai phá.

F. Ablation Study

Để làm rõ đóng góp của từng thành phần, chúng tôi tiến hành phân tích tác động của cơ chế *Adaptive maxPattern-Length*. Hình 1 mô phỏng mức tiêu thụ bộ nhớ dưới các đợt lưu lượng dữ liệu tăng đột biến (traffic bursts). Nhờ khả năng tự động cắt giảm L_{max} khi tài nguyên cạn kiệt, hệ thống luôn duy trì mức bộ nhớ cực kỳ thấp và ổn định. Ngược lại, nếu tắt tính năng này (Fixed length), mô hình rất dễ gặp lỗi tràn bộ nhớ (Out-Of-Memory) trên dòng dữ liệu vô hạn.

Đồ thị Mức tiêu thụ bộ nhớ theo thời gian

Hình 1: Cơ chế Adaptive giúp duy trì mức sử dụng bộ nhớ luôn ổn định và an toàn.

G. Scalability Analysis

Tính khả thi trên dòng dữ liệu thực tế đòi hỏi khả năng mở rộng (scalability) cao. Khi kiểm thử trên tập dữ liệu REDD đầy đủ (với 767.829 sự kiện), BFSPMiner-Improved thể hiện mức tăng trưởng thời gian chạy theo ****tuyến tính**** tuyệt đối. So

với Baseline, chúng tôi không gặp phải hiện tượng tắc nghẽn cổ chai hay bùng nổ thời gian theo hàm mũ, qua đó chứng minh thuật toán có khả năng hoạt động liên tục trong môi trường công nghiệp 24/7.

VI. THẢO LUẬN

Kết quả thực nghiệm đã làm nổi bật tác động to lớn của việc nối lỏng các ràng buộc cứng nhắc trong bài toán khai phá luồng dữ liệu. Bằng cách trang bị cơ chế độ dài thích ứng và khả năng nhận diện các sự kiện có khoảng trống, hệ thống không chỉ thu thập được một lượng tri thức phong phú hơn đáng kể (+70% số mẫu) mà còn tối ưu hóa tài nguyên phần cứng.

While a direct comparison with recent state-of-the-art streaming algorithms is left for future work, our method significantly outperforms the original BFSPMiner baseline in both pattern completeness and prediction quality. (Mặc dù việc so sánh trực tiếp với các thuật toán khai phá luồng hiện đại gần đây sẽ được thực hiện trong các nghiên cứu tương lai, phương pháp của chúng tôi đã vượt trội đáng kể so với phiên bản BFSPMiner gốc cả về mức độ hoàn chỉnh của tập mẫu lẫn chất lượng dự đoán).

Sự cải tiến này đặc biệt hữu ích cho các kịch bản thời gian thực như phân tích hành vi clickstream, nơi mà hành vi của người dùng hiếm khi diễn ra liên tục theo một trật tự hoàn hảo mà thường bị ngắt quãng bởi các tác vụ phụ.

VII. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Bài báo đã đề xuất BFSPMiner-Improved, một thuật toán khai phá mẫu tuần tự không theo lô (batch-free) có khả năng thích ứng cao dành riêng cho luồng dữ liệu liên tục. Thông qua việc phát triển cơ chế thích ứng độ dài động và mở rộng khung khai phá tập sự kiện hỗ trợ khoảng trống, thuật toán đã giải quyết thành công các hạn chế của mô hình gốc, xử lý tốt cả những dữ liệu IoT có cấu trúc lẫn luồng clickstream nhiều nhiễu.

Các đánh giá thực nghiệm nghiêm ngặt đã xác nhận tính ưu việt của phương pháp so với các Baseline nền tảng. Hệ thống không chỉ tăng thêm 70% số mẫu phát hiện được mà còn đạt Precision@3 tới 0.984, đồng thời đảm bảo khả năng mở rộng tuyến tính và kiểm soát tốt bộ nhớ. Hướng nghiên cứu trong tương lai sẽ tập trung vào việc song song hóa các quy trình cập nhật cây Trie và tích hợp vào các hệ sinh thái xử lý phân tán như Apache Flink để triển khai khai phá quy mô lớn trên các cụm máy chủ.